End-to-End Product Workflow — User Guide

See your product before you build. Scope deliberately. Scale on a plan that holds up.

From idea to shipped product, end-to-end.

> **IMPORTANT**
>
> The prototype is **notional**. It's a directional reference for shaping decisions, not a pixel-perfect spec. Due to technical constraints or branding compliance, Claude Code may not be able to build it exactly as designed. Set expectations with reviewers and stakeholders accordingly.
>
> *Example: if you change a font or color in Claude Design, it might be changed in the `/build` phase to align with the design system.*

> **A note on file paths.** Claude Code creates and manages the folder structure for you. The only time you'll touch files yourself is in **Step 7**, when you save the prototype's export into your project.
>
> **A note on screenshots.** Claude Design's UI gets updated regularly, so the screenshots in this guide may look slightly different from what you see. The flow stays the same — look for the same buttons and options even if their styling has changed.

Quick steps

1. **Set up a new application** — in Windows PowerShell or Terminal, install the project template and create your app folder. Run this before every new project so you get the latest template.
2. **Launch Claude Desktop** and point Claude Code at your app folder. Go to the **Code tab** and change the working folder to the folder you created in Step 1.
3. **Describe your product** to `/requirements-gathering` in Claude Code, then run `/requirements-ship` to sync the doc to its main location. The first command asks follow-up questions and writes up the requirements; the second syncs it to where the later skills read it from.
4. **Run `/design-foundation` in Claude Code**, then review and sign off on the proposed selection. It reads your requirements and decides which screens to prototype.
5. **Build screens one at a time** in Claude Design. Select the McDermott design system, paste the design brief, and steer through screens in flow order.
6. **Share for feedback** *(optional)* — export the prototype from Claude Design and refine based on what you hear back.
7. **Manually move the project archive** (Claude Design's full export — not the HTML-only or single-screen options) from Claude Design into your project folder. The next step's skill can't fetch it for you — this is the one manual file move in the workflow.
8. **Run `/design-code-handoff` in Claude Code**, then review and sign off on what changed. It reconciles the blueprint against what was actually built.
9. **Run `/plan` in Claude Code** to start the build pipeline (or hand the project to your developer). Stay available to review what gets built.
10. **Run `/build` in Claude Code** to implement the build plan. Stay available to review what gets built.
11. **Run `/ship` in Claude Code** to finalize and deploy the product.
12. **Test the app locally** *(Build-a-thon temporary)* — copy the OS-specific prompt into Claude Code to launch the frontend, API, and database locally.

That's the whole workflow. The details for each step are below.

🧰 Step 1 — One-time setup for a new application

Windows PowerShell or Terminal

> **YOUR PART —** *Install the project template and create your app folder.*

1. Install the latest solutions template

Run this before every new project so you get the latest template. In Windows PowerShell or Terminal, copy-paste and run one command at a time:

```
npm cache clean --force
npm install -g @alan-eicker/northstar-cli
```

> **Note:** If you get an execution policy error, run this command first:
>
> ```
> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
> ```

2. Create your APP folder

Navigate to `C:\Users\[user_name]\` in File Explorer and create a new folder named `claude_apps` if it doesn't exist already. Replace `[user_name]` with your username on the machine — check `C:\Users\` to identify this.

Run these two commands in Windows PowerShell or Terminal. Replace `[user_name]` and `[app_name]` with your actual values:

```
cd C:\Users\[user_name]\claude_apps
nstar create [app_name]
```

You'll get prompted with a few options:

- **GitHub account type?** Enter `1` (Personal).
- **GitHub access token?** Paste the token you created during the one-time machine config (a prerequisite setup step outside this guide).

> **Note:** The token won't appear in the prompt for security reasons — it will look empty. Just paste and hit Enter.

After this step, a folder named `[app_name]` should exist at `C:\Users\[user_name]\claude_apps\`. Confirm manually using File Explorer.

💻 Step 2 — Open your project in Claude Code

Claude Desktop

> **YOUR PART —** *Launch Claude Desktop and point Claude Code at your app folder.*

Launch **Claude Desktop**, go to the **Code tab**, and change the working folder to point to the folder you created in **Step 1**.

📝 Step 3 — Gather your requirements

Claude Code · requirements-gathering + requirements-ship

> **YOUR PART —** *Run `/requirements-gathering` to describe your product, then `/requirements-ship` to sync the doc.*

1. Within your project in Claude Code, run **`/requirements-gathering`**. Describe your product to Claude. It'll ask follow-up questions and produce **`solution-requirements.md`** in a temporary worktree folder.
2. Run **`/requirements-ship`** to sync the doc from the temporary worktree folder to `artifacts/docs/product/` (its main location).

You'll use that file in Steps 4 and 8 — both skills read it from there automatically.

> **Tip:** if Claude asks something you can't answer yet, pause and think. Guesses harden into product decisions.

📋 Step 4 — Create a blueprint

Claude Code · design-foundation

> **YOUR PART —** *Sign off on the proposed selection.*

1. Within your project in Claude Code, run `/design-foundation`. It will read the requirements document to create the handoff for Claude Design and the design blueprint.
2. Review the proposed selection — which screens are in the prototype, which are deferred.
3. **Sign off** when you're confident — type something like "*looks good, proceed*" or "*approved*" to move forward. Or push back if anything feels off.

The skill writes two files to `artifacts/docs/design/`: `full-design-blueprint.md` and `HANDOFF-design-brief.md`. When it's done, **it gives you a download link for the HANDOFF brief** — save the file to your computer so you can upload it into Claude Design in the next step.

> **Tip:** the selection is the most important decision in the workflow. Push back at this checkpoint if anything feels off.

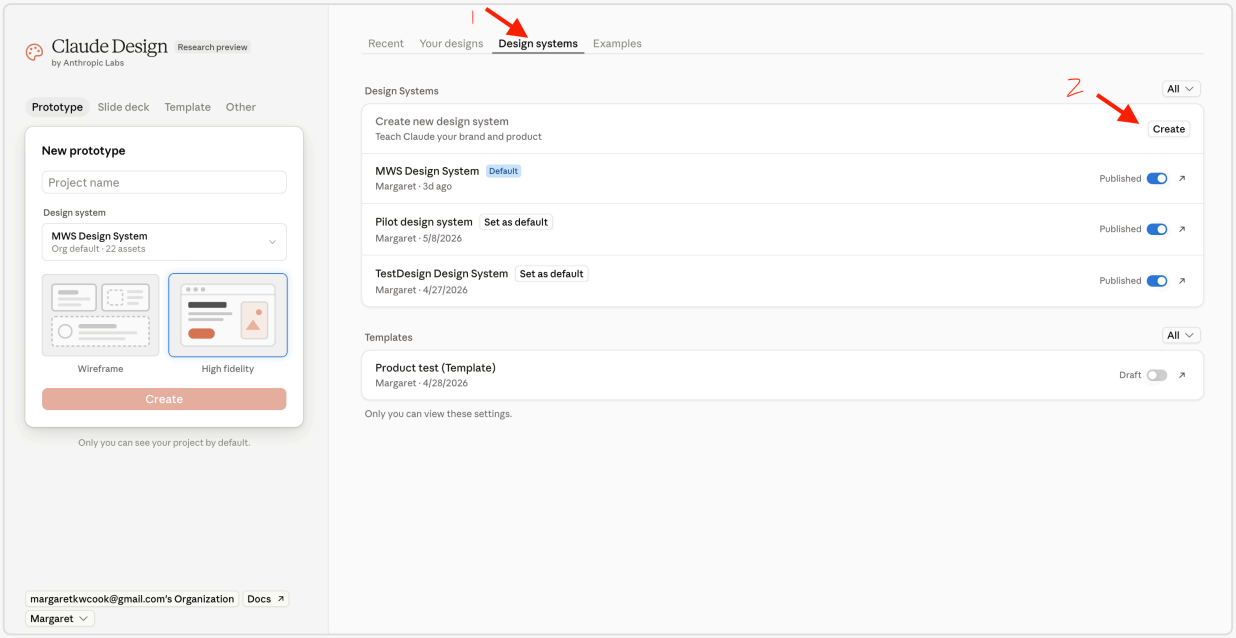
⚙️ Setup — design system in Claude Design *(one-time, if needed)*

Claude Design

> **YOUR PART —** *Set up the McDermott design system in Claude Design (if it's not already there).*

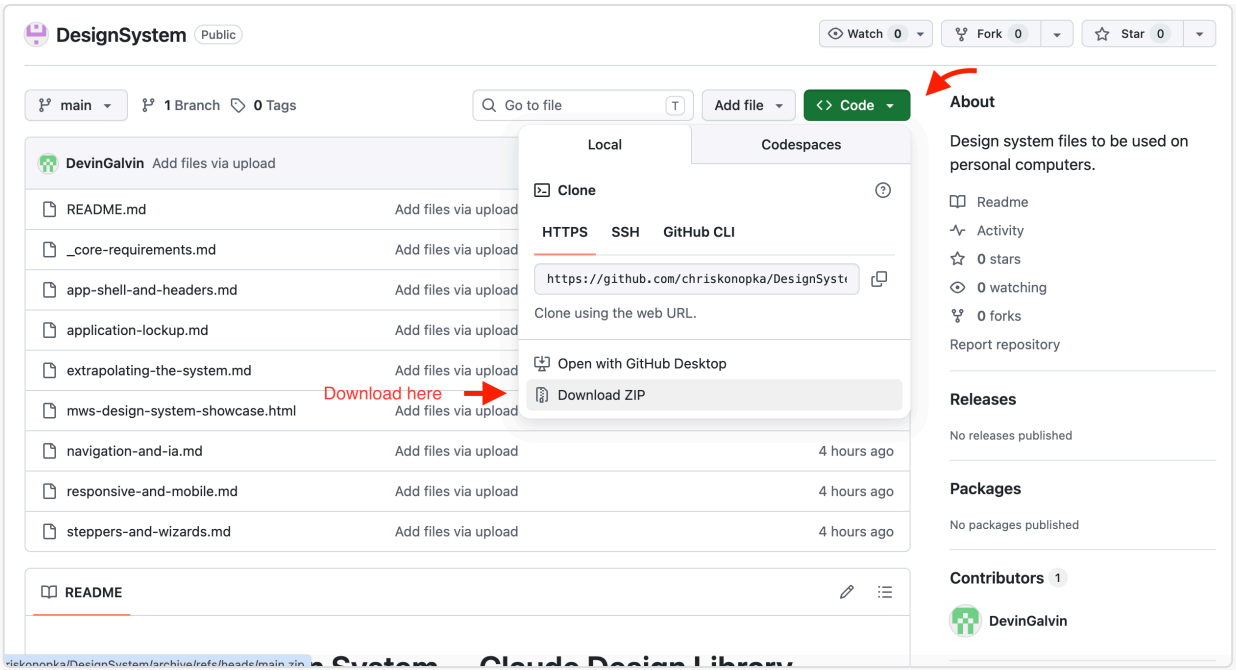
The McDermott design system is the foundation Claude Design uses to build your prototype. The more complete and polished it is, the better the prototype will look — and the closer your final build will be to it.

- Open **Claude Design** (claude.ai/design) and check your design systems list.



Open Claude Design and find the design systems list.

- If the McDermott design system is already there, skip to **Step 5 — Build the prototype**.
- If it's not there, download the latest **McDermott design system file package** from GitHub.



GitHub download: navigate the McDermott design system repo and download the files.

- **Unzip the downloaded folder**, then upload the files into Claude Design through the **"Link code on your personal computer"** option to set up the design system.

> **Tip:** Some users have had trouble uploading the files directly from their Downloads folder. If you run into issues, move the unzipped folder to your desktop (or another location on your machine) first.

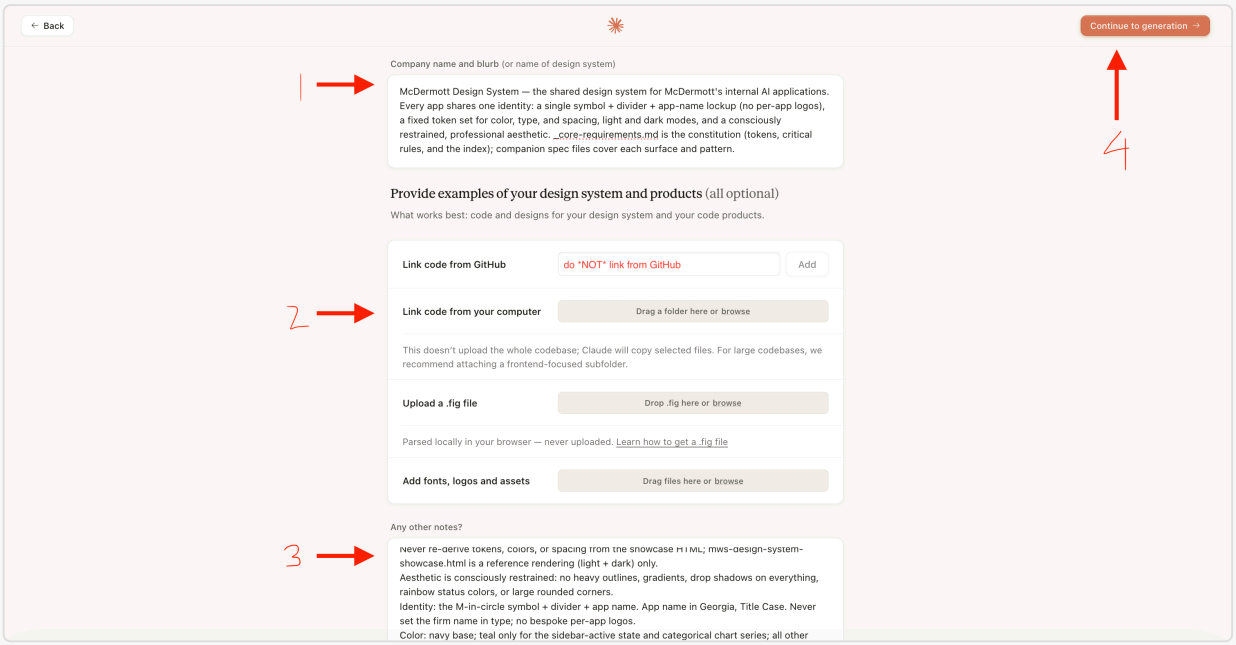
In the "Company name and blurb (or name of design system)" field, paste:

> McDermott Design System — the shared design system for McDermott's internal AI applications. Every app shares one identity: a single symbol + divider + app-name lockup (no per-app logos), a fixed token set for color, type, and spacing, light and dark modes, and a consciously restrained, professional aesthetic. `_core-requirements.md` is the constitution (tokens, critical rules, and the index); companion spec files cover each surface and pattern.

In the "Any other notes?" field, paste:

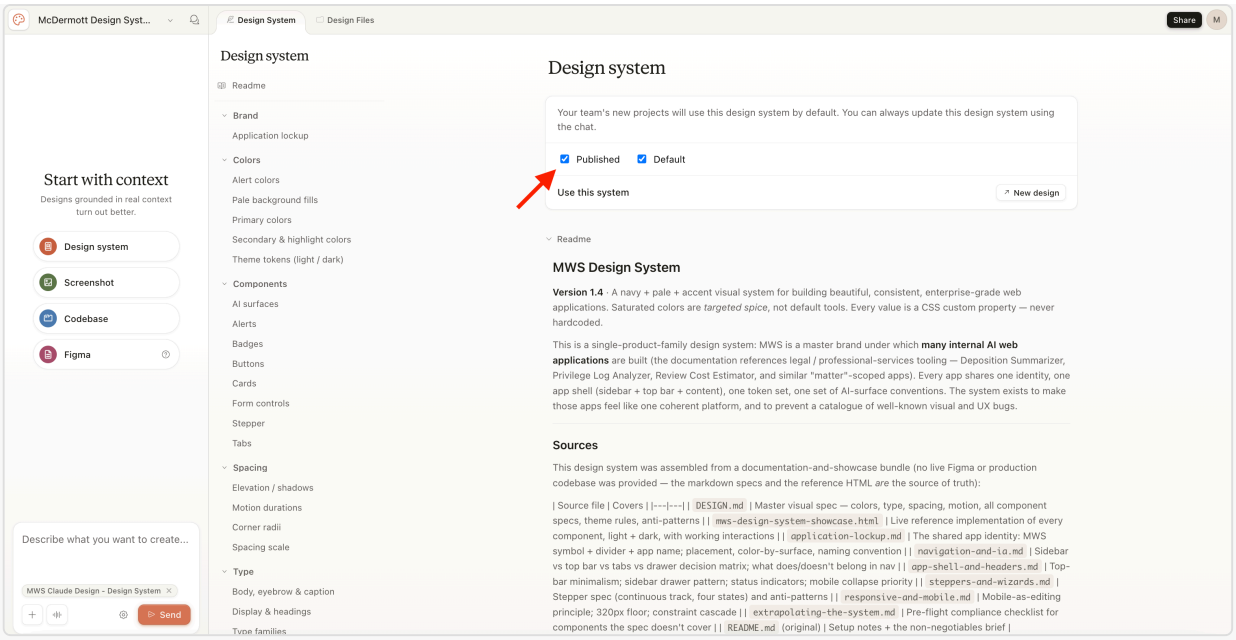
> - The attached folder is the single source of truth. Load `_core-requirements.md` (the constitution) plus the relevant companion specs — small focused files, not one mega-doc.
> - Never re-derive tokens, colors, or spacing from the showcase HTML; `mws-design-system-showcase.html` is a reference rendering (light + dark) only.
> - Aesthetic is consciously restrained: no heavy outlines, gradients, drop shadows on everything, rainbow status colors, or large rounded corners.

- Identity: the M-in-circle symbol + divider + app name. App name in Georgia, Title Case. Never set the firm name in type; no bespoke per-app logos.
- Color: navy base; teal only for the sidebar-active state and categorical chart series; all other interactive elements use the accent (blue in light, teal in dark). Both themes required.
- Accessibility target: WCAG 2.1 AA. Icons: Phosphor, outline only.



Set up the McDermott design system in Claude Design by uploading the files.

- Wait for the design system to finish setting up. This may take up to 5 minutes.
- **Publish the design system** to make it available for use in your projects. Without publishing, the system won't show up when you go to build a prototype.



Publish the design system in Claude Design to make it available for use.

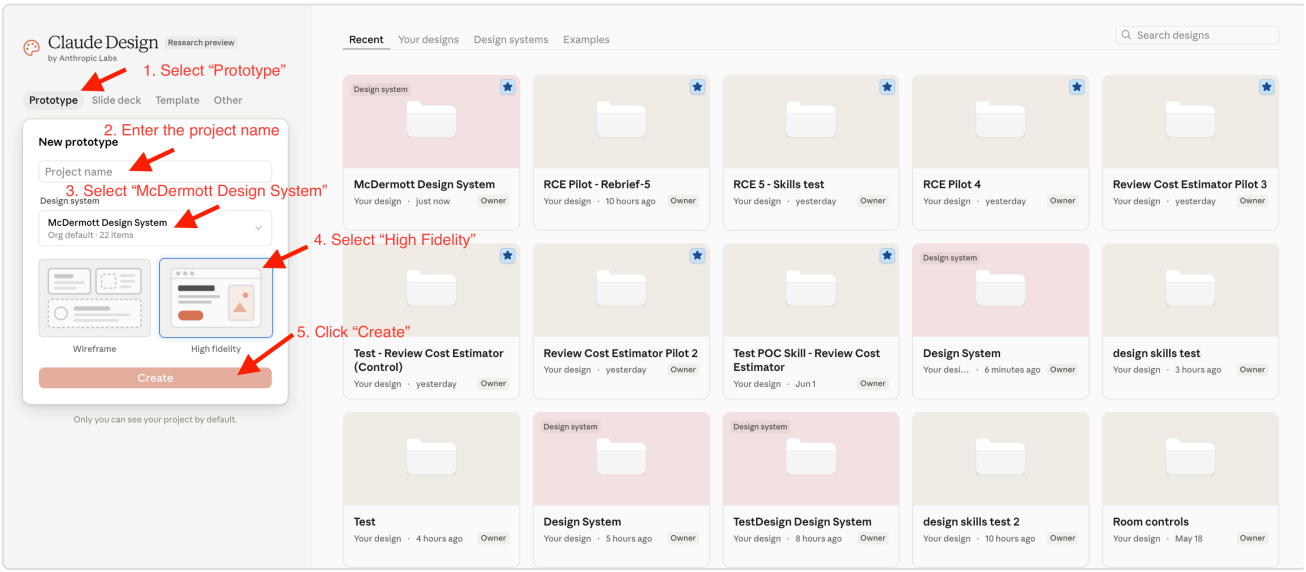
> **Tip:** a thorough McDermott design system is the biggest factor in a clean prototype. A thin system (just a logo and a color) produces improvised-looking output. Time invested here pays off at every later step.

🎨 Step 5 — Build the prototype

Claude Design

> **YOUR PART —** *Build screens one at a time in Claude Design.*

1. In **Claude Design**, enter the product name, select "High Fidelity" and the McDermott design system, then click "Create".



Enter your product name, toggle "High Fidelity", and select the McDermott design system before clicking Create.

2. Upload the `HANDOFF-design-brief.md` file you downloaded in Step 4, then click "Send".
3. **Build screen 1 first.** When you're happy with it, **prompt Claude to build the next one** by saying something like "*now build screen 2*". Continue prompting for each new screen — Claude builds one at a time and won't move forward on its own.
4. Refine any screen by commenting on it, editing text directly, or using the tweaks panel.

> **Tip:** It's fine to build only some of the screens, not all of them. If you stop early — because you ran out of time, decided a screen isn't worth prototyping, or just want to ship what you have — design-code-handoff in Step 8 will catch the unbuilt screens and ask you what to do with each one: prototype it later, build it from the blueprint instead, or drop it entirely. You decide screen by screen.

💬 Step 6 — (Optional) Share for review

Claude Design

> **YOUR PART —** *Export the prototype and collect feedback.*

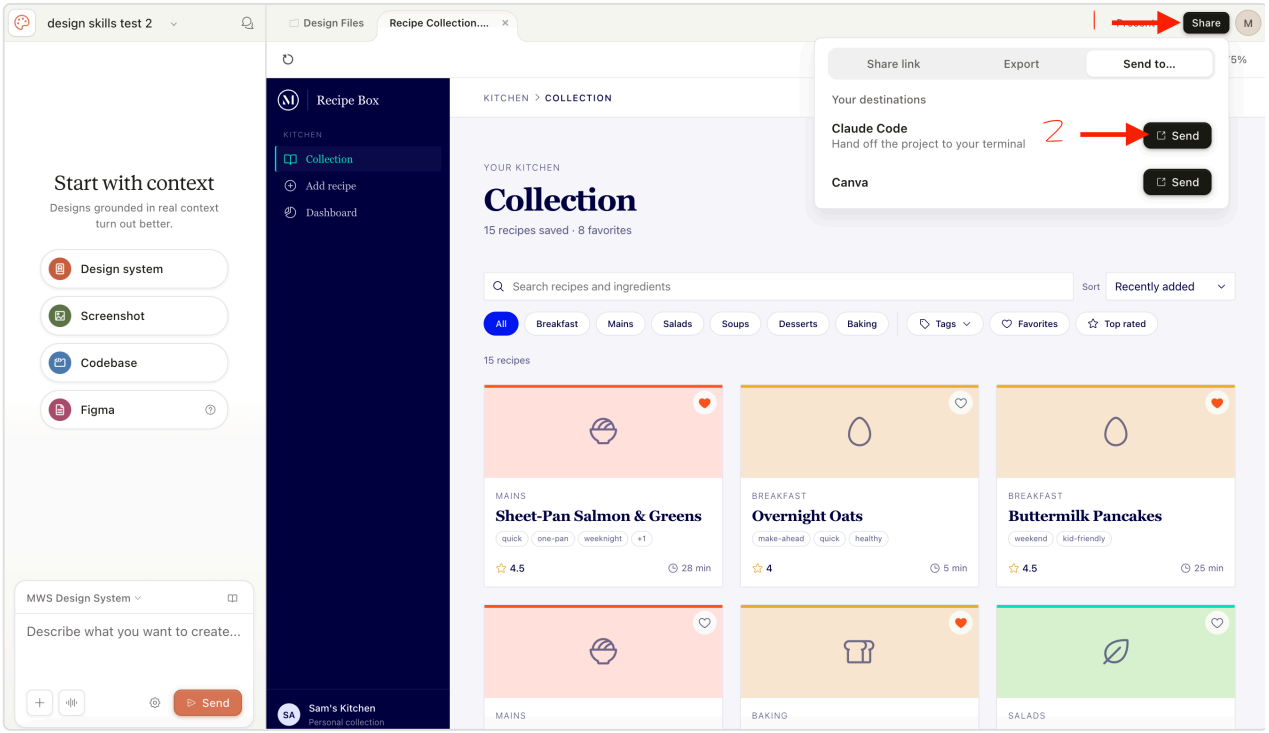
Export the prototype as a shareable URL, HTML, PDF, or PPTX. Collect feedback. Refine in Claude Design and re-export until you're happy.

📦 Step 7 — Export the .zip and add it to your project

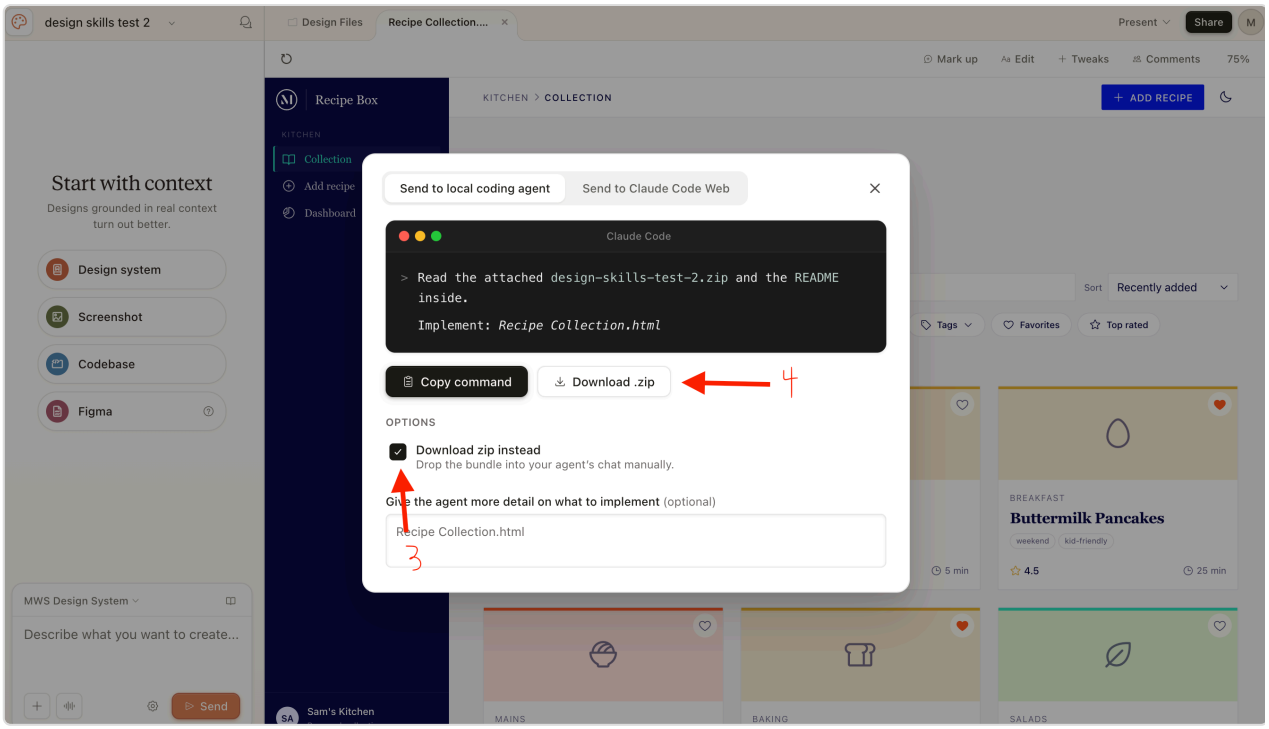
Claude Design + your computer

> **YOUR PART —** *Move the prototype .zip into your project.*

1. In **Claude Design**, export the approved prototype as a project archive (a `.zip` of the full package — not the HTML-only export or a single-screen export). Three ways to do this — **Option A is the preferred path**:
   - ⭐ **Option A (preferred) — Share with Claude Code:** From the **share menu**, choose **"Send to..."** → **Claude Code**, then in the modal that opens, check **"Download zip instead"** and click **"Download .zip"**. **Do NOT** send the project straight to the Claude Code chat — the skill needs the archive saved to your `artifacts/docs/design/` folder, not piped into a chat conversation.

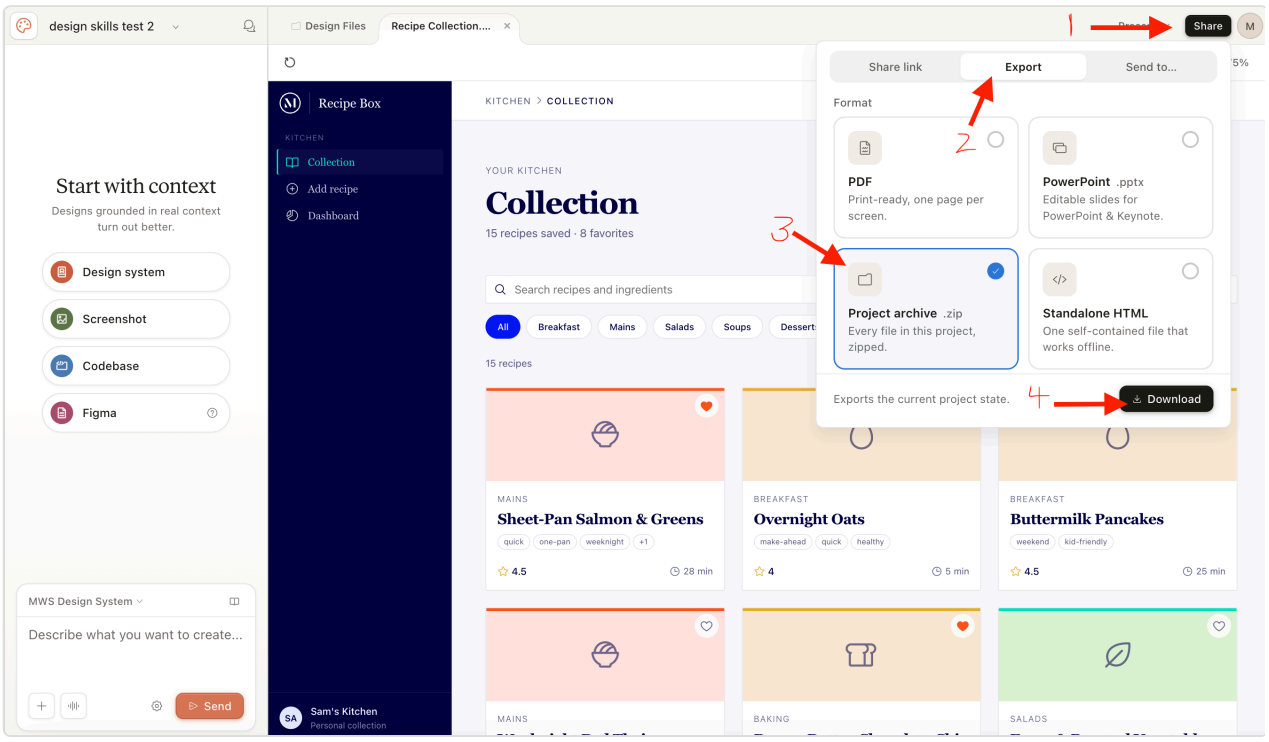


Steps 1-2: Click the Share button, open the "Send to..." tab, then click Send on Claude Code.



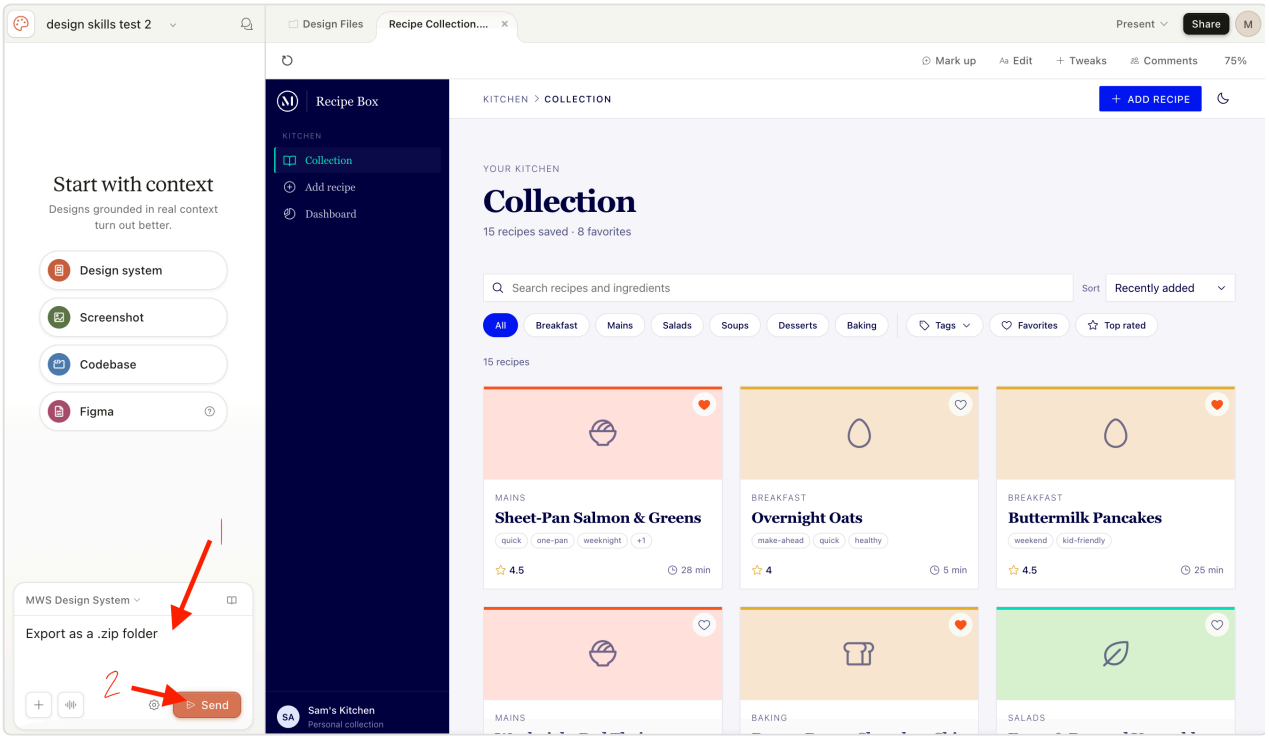
Steps 3-4: Check "Download zip instead," then click "Download .zip."

- **Option B — Share menu:** Click the **share button** in the top-right corner → **Export** → choose **"Project archive .zip"** → **Download**.

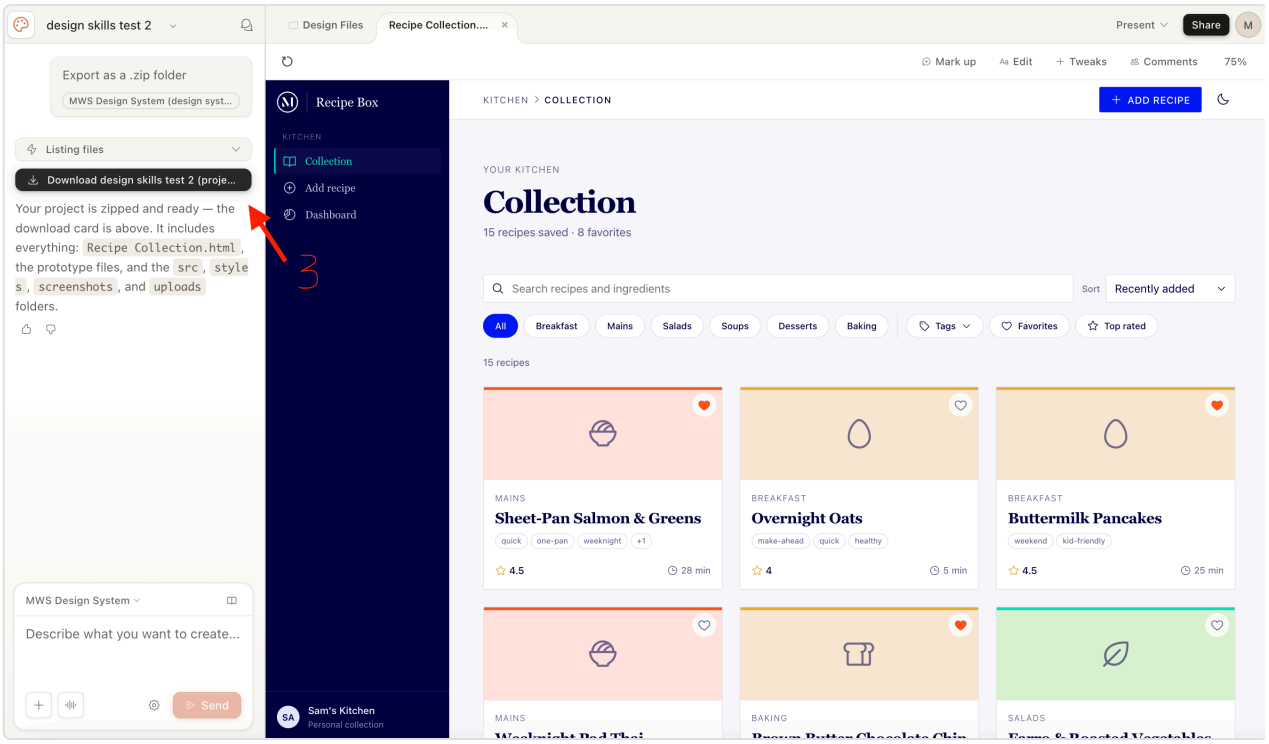


Click Share, switch to Export, choose "Project archive .zip," then click Download.

- **Option C — Ask in chat:** In the **chat panel** on the left, type "*I would like to export this project as a .zip folder*", click **Send**, then download the zip from Claude Design's response.



Steps 1-2: Type your request in the chat panel, then click Send.



Step 3: Download the zip from Claude Design's response.

2. On your computer, move or copy the downloaded project archive into `artifacts/docs/design/` — the same folder where `full-design-blueprint.md` already lives.

> **Tip:** keep the project archive's folder structure intact (don't unzip or rearrange it). The next step's skill will rename the file to `project/` automatically so every project uses the same canonical path. Claude Design typically packages a few different versions of each screen inside the archive (and may also produce loose HTML files from previous Step 6 sharing) — the skill identifies which one is the right version for the build using the file's structure, so no cleanup is needed on your end.

✅ Step 8 — Reconcile the plan with the prototype

Claude Code · design-code-handoff

> **YOUR PART —** *Sign off on what changed.*

1. Run `/design-code-handoff` from your active directory.
2. **Sign off** on the differences the skill found:
   - **Added** — things in the design that weren't in the original plan.
   - **Missing** — things in the plan that aren't in the design (including screens you planned to prototype but didn't build — you'll decide per screen whether to prototype later, shift to the blueprint, or drop).
   - **Changed** — things present in both but looking different.
   - **Tier-gate violations** — where the design exceeds the rules for your product's tier (fix the design to fit, or promote to a higher tier).

   Push back on any finding you disagree with.
3. After your sign-off, the skill updates the blueprint (and `solution-requirements.md`, if you accepted scope changes or a tier promotion). Nothing moves — the skill updates the files where they already live.

The design brief (`HANDOFF-design-brief.md`) is left alongside but is superseded by the prototype.

> **Tip:** you don't need to remember or describe what changed during prototyping. The skill reads the prototype directly.

🚀 Step 9 — Plan the build

Claude Code

> **YOUR PART —** *Run `/plan`.*

In **Claude Code**, run `/plan` to start the build pipeline. It reads everything in place — your reconciled blueprint, requirements doc, and prototype export folder — and produces the build plan that drives the rest of the implementation.

Stay available to review.

🛠️ Step 10 — Build the product

Claude Code

> **YOUR PART —** *Run `/build`.*

In **Claude Code**, run **`/build`** to implement the build plan from Step 9. It writes the code for your product, screen by screen.

Stay available to review.

🚢 Step 11 — Ship the product

Claude Code

> **YOUR PART —** *Run `/ship`.*

In **Claude Code**, run `/ship` to finalize and deploy your product.

Stay available to review.

🧪 Step 12 — Test the app locally

Claude Code

> **YOUR PART —** *Copy the prompt for your operating system and paste it into Claude Code.*

> **Note:** This is a temporary solution for the Build-a-thon. Claude Code will start the frontend and API services, create a database, and launch the web app so you can view it locally.

Windows — copy this prompt into Claude Code:

```
Launch the app locally for testing.

My environment:
- .NET SDK 10, MS SQL Server 2025 LocalDB
- Frontend: React (TypeScript/Vite), Middle tier: .NET Web API, Database: LocalDB

1. Confirm or set up the LocalDB connection string in appsettings.Development.json
2. Apply any pending database migrations
3. Start the .NET API and React frontend (restore packages if needed)
4. Add a dev-only Entra auth bypass:
   - Skip token validation in the API when ASPNETCORE_ENVIRONMENT=Development
   - Inject a mock user so all API endpoints work without a real token
   - Skip MSAL sign-in in the React frontend and go straight to the app
5. Tell me the localhost URLs for both once running

Rules:
- Auth bypass must never activate outside Development
- Do not commit or push any of these local testing changes to dev
- Add all bypass/test files to .gitignore
```

Mac — copy this prompt into Claude Code:

```
Launch the app locally for testing.

My environment:
- .NET SDK 10, SQL Server running in a Docker container (Mac)
- Frontend: React (TypeScript/Vite), Middle tier: .NET Web API, Database: SQL Server
container

1. Check if the SQL Server Docker container is running, start it if not
2. Confirm or set up the connection string in appsettings.Development.json pointing
to the container
3. Apply any pending database migrations
4. Start the .NET API and React frontend (restore packages if needed)
5. Add a dev-only Entra auth bypass:
   - Skip token validation in the API when ASPNETCORE_ENVIRONMENT=Development
   - Inject a mock user so all API endpoints work without a real token
   - Skip MSAL sign-in in the React frontend and go straight to the app
6. Tell me the localhost URLs for both once running

Rules:
- Auth bypass must never activate outside Development
- Do not commit or push any of these local testing changes to dev
- Add all bypass/test files to .gitignore
```

Once Claude finishes, it will give you the localhost URLs for the frontend and API. Open the frontend URL in your browser to view the app.

If something goes wrong

- **The prototype is cluttered or off-brand:** add more of the McDermott design system to Claude Design and rebuild.
- **The prototype diverged from the plan:** that's fine. design-code-handoff handles it.
- **You have questions during the build:** answers should be in `full-design-blueprint.md`. If they're not, the gap is in the plan — go back and add detail.
- **design-code-handoff says it can't read the prototype:** the project archive's HTMLs may all be expired files or placeholders. To recover:
  1. Open `artifacts/docs/design/`.
  2. Delete the project archive (`.zip` and any `project/` folder; keep the blueprint and brief).
  3. In Claude Design, export the most recent iteration as a standalone HTML (Share menu → Export → Standalone HTML, or ask in the chat panel).
  4. Drop the standalone HTML into `artifacts/docs/design/`.
  5. Run design-code-handoff again.

Quick reference

Step	What you do	What you get
🧰 1. Set up a new application	Install the project template + create app folder in PowerShell / Terminal	App folder ready at `C:\Users\[user_name]\claude_apps\[app_name]\`
💻 2. Open your project in Claude Code	Launch Claude Desktop; point Claude Code at your app folder	Working folder set
📝 3. Gather requirements	Run **`/requirements-gathering`** + **`/requirements-ship`** in Claude Code	`solution-requirements.md` in `artifacts/docs/product/`
📋 4. Create a blueprint	Run **`/design-foundation`**; sign off on the selection	`full-design-blueprint.md` + `HANDOFF-design-brief.md`
⚙️ Setup. Design system *(if needed)*	Set up the McDermott design system in Claude Design	Design system ready to use
🎨 5. Build the prototype	Drive Claude Design to build screens one at a time	Approved prototype
💬 6. Share for review *(optional)*	Export the prototype and collect feedback	Refined prototype
📦 7. Export the prototype folder	Save the project archive into `artifacts/docs/design/`	Prototype folder ready for reconciliation
✅ 8. Reconcile with prototype	Run **`/design-code-handoff`**; sign off on changes	Reconciled blueprint
🚀 9. Plan the build	Run **`/plan`** (or hand off to a developer)	Build plan
🛠️ 10. Build the product	Run **`/build`** in Claude Code	Implemented product
🚢 11. Ship the product	Run **`/ship`** in Claude Code	Deployed product

Step	What you do	What you get
🧪 12. Test the app locally *(Build-a-thon temporary)*	Paste the OS-specific prompt into Claude Code	App running locally on your machine

Three Claude Code skills, three build commands, one prototype tool. You're shaping decisions at every step — Claude handles the structured work in between.